Event Based Control—Project Proposal

Supervisory Control of an Automatic Espresso Machine
with Fault Handling and Service Actions

October 15, 2026

Name	Index
Bartosz Ksel (coordinator)	275472
Krzysztof Pogoda	275431
Maciej Smolarz	275491
Polona Umek	300468

1 Problem

The subject of the project is the supervisory, event-based controller of a fully automatic espresso machine of the kind used at home and in small offices. The machine consists of a bean grinder, a single boiler that works either at brewing temperature or at steam temperature, a water pump, a brewing unit, a milk pump with a steam frother, and a set of sensors: water tank level, bean presence, milk presence, grounds container counter, drip tray level, brewing unit presence and boiler temperature.

At the supervisory level the machine is a typical discrete event system. Its behaviour is driven by user requests (a drink is ordered, a cup is placed), by sensor events (temperature reached, water tank empty, grounds container full) and by service actions performed by the user (refilling, emptying, descaling). The controller has to decide which of its own actions (heating, grinding, brewing, steaming, rinsing, blocking a drink) may be executed in the current state, and in which order.

The topic is worth taking up for three reasons. First, the controller must be *safe*: an actuator must never run in a condition that can damage the machine or the user, e.g. the pump without water or the steamer without milk. Second, it must be *recoverable*: after any combination of faults the machine has to return to normal operation once the corresponding service actions are performed, and must never get stuck. Third, the single boiler and the shared hydraulic path create genuine resource conflicts, so the order of operations influences the drink preparation time and the number of temperature changes, i.e. the efficiency of the machine.

An important design goal is *graceful degradation*: a fault blocks only the drinks that actually depend on the missing resource. For example, an empty milk container blocks cappuccino but not espresso, while a full grounds container blocks grinding but lets a drink that is already brewing finish.

Expected result. The project will deliver (i) a formal model of the machine (a Petri net, see Section 2.7) together with the rules EF(s), next_state(s, e) and safe(s, e), (ii) an analysis demonstrating safety and recoverability of the model, (iii) an event-driven simulator of the machine with fault and service scenarios, and (iv) a comparison of event prioritization policies with respect to the efficiency criteria defined in Section 2.6.

2 Model and Control Concept

The menu is limited to three drinks that together exercise every resource: *espresso* (grinder, brew unit, boiler at brew temperature), *americano* (espresso plus hot water) and *cappuccino* (espresso plus milk, boiler at steam temperature). At most one drink is prepared at a time; a request queue can be added later without changing the structure of the model.

2.1 System state

The state is a tuple $s = (m, r, c, d, F, p) \in S$, where

- $m \in \{\mathit{off}, \mathit{heating}, \mathit{ready}, \mathit{preparing}, \mathit{fault}, \mathit{service}, \mathit{descaling}\}$ is the operating mode;
- r is the state of the resources: grinder, pump and steamer $\in \{\mathit{idle}, \mathit{busy}\}$, boiler $\in \{\mathit{cold}, \mathit{brew}, \mathit{steam}\}$, brewing unit $\in \{\mathit{empty}, \mathit{loaded}, \mathit{used}\}$, cup $\in \{\mathit{absent}, \mathit{present}\}$;
- c are the consumables: water $\in \{\mathit{ok}, \mathit{low}, \mathit{empty}\}$, beans and milk $\in \{\mathit{ok}, \mathit{empty}\}$, grounds counter $g \in \{0, \ldots, N_g\}$, drip tray $\in \{\mathit{ok}, \mathit{full}\}$;
- $d \in \{0, \ldots, N_{\text{over}}\}$ is the descaling counter (drinks since the last descaling), with a warning threshold $N_{\text{due}} < N_{\text{over}}$;
- $F \subseteq \mathcal{F}$ is the *set* of active faults, since several faults may be active at once;
- p is the current order: the requested drink and the stage of its recipe, or $\varnothing$.

The initial state s_0 is: mode *off*, boiler *cold*, all resources idle, consumables full, $g = d = 0$, $F = \varnothing$, $p = \varnothing$. The machine runs continuously, so instead of a final state we use the *home state* s_h: mode *ready*, no order, no faults.

2.2 Events

- E_{obs} (uncontrollable, generated by the user and the sensors): `power_on`, `order_espresso`, `order_americano`, `order_cappuccino`, `cup_placed`, `cup_removed`, `temp_reached`, `grind_done`, `brew_done`, `steam_done`; faults: `water_low`, `water_empty`, `beans_empty`, `milk_empty`, `bin_full`, `tray_full`, `brew_unit_removed`, `flow_error`, `overheat`; service actions: `refill_water`, `refill_beans`, `refill_milk`, `empty_bin`, `empty_tray`, `insert_brew_unit`, `descaling_confirmed`.
- E_{con} (decided by the controller): `heat_to_brew`, `heat_to_steam`, `start_grind`, `start_brew`, `add_hot_water`, `start_steam`, `eject_grounds`, `start_rinse`, `start_milk_cleaning`, `start_descaling`, `abort_drink`, `reject_order`.

2.3 Events possible in a state

Each drink k has a set of required resources $R(k)$, and each fault $f \in \mathcal{F}$ a set of resources it disables, $B(f)$. A drink is *available* in s if $R(k) \cap \bigcup_{f \in F} B(f) = \varnothing$ and $d < N_{\text{over}}$. Then:

- an order event for drink k is in EF(s) iff $m = \textit{ready}$ and k is available; otherwise only `reject_order` follows;
- each controllable event has a guard on the state; for example `start_brew` $\in$ EF(s) iff boiler $=$ *brew*, brew unit $=$ *loaded*, cup $=$ *present*, water $\neq$ *empty* and pump $=$ *idle*; `start_grind` $\in$ EF(s) iff beans $=$ *ok*, $g < N_g$ and the brew unit is empty;
- fault events are always possible (the environment is not controlled), service events are possible when the corresponding fault is active.

2.4 State transition

next_state(s, e) is defined event by event as an update of the tuple. Starting an operation marks the resource as *busy* and advances the recipe stage of p; the matching completion event (`grind_done`, `brew_done`, ...) releases the resource. Completing a drink increments g and d and returns to *ready*. A fault event adds f to F; if it affects the drink in preparation, the mode becomes *fault* and the running operation is stopped, otherwise the mode is unchanged. A service event removes f from F, and when $F = \varnothing$ the machine returns to *ready*. In the Petri net representation these rules are exactly the firing rule of the net.

2.5 Correctness and safety

Two qualitative criteria will be demonstrated:

1. **Safety.** No reachable state allows an unsafe actuation: the pump never runs without water or a cup, the steamer never runs without milk, the grinder never runs with a full grounds container, no actuator runs while `overheat` is active, and a drink is never started unless all resources in $R(k)$ are available. This is enforced by safe(s, e) $\in \{\text{true}, \text{false}\}$, which checks the actuator conditions in addition to EF(s).
2. **Recoverability (liveness).** From every reachable state, the home state s_h is reachable once the user performs the service actions for the active faults; i.e. s_h is a home marking of the net and the net is deadlock-free.

Both properties will be checked on the reachability graph of the model (which is finite, since all counters are bounded) and supported by P-invariants of the net [2]. A special case is a fault in the middle of a drink, e.g. `milk_empty` during a cappuccino: the controller finishes the espresso part, reports a partial drink and must not block the machine.

2.6 Event prioritization and efficiency

The efficiency criteria are the drink preparation time, the number of boiler temperature changes, and the availability of the machine (the fraction of time in which it can serve a drink). Conflicts appear when several controllable events are enabled at once:

- order of brewing and steaming in a cappuccino (one boiler, one temperature switch per drink either way, but different heating times);
- running heating and grinding in parallel instead of sequentially;

- when to run descaling once it is *due*: immediately, or only after a period of inactivity, so that it does not delay a request;
- rinsing after every milk drink or after a batch.

Each rule is expressed as a priority order on $E_{\text{con}} \cap \text{EF}(s)$. We will compare at least a fixed-order baseline with an optimised policy in simulation with randomly generated orders and faults.

2.7 Proposed modelling approach

We choose **Petri nets** [2, 1]. Resources map naturally to places with tokens (the grounds container capacity is a place with N_g tokens), concurrent activities such as heating and grinding are represented without a combinatorial explosion of states, faults are places that inhibit transitions, and the required properties (boundedness, deadlock freedom, home marking) have standard analysis methods. A plain finite automaton would require an explicit product of all component states and would make concurrent faults hard to represent.

2.8 Planned implementation

The model will be built and analysed in a Petri net tool (TINA or PIPE). The controller and the plant will be implemented in Python as a discrete event simulator: the plant generates observable events (orders, sensor events, random faults and service actions), and the controller selects controllable events using $\text{EF}(s)$, $\text{safe}(s, e)$ and the prioritization policy. A simple GUI with buttons for user and fault events will be used for the demonstration. Code and documents are kept in a shared Git repository.

3 Workplan

The schedule follows the class calendar (Tuesdays, 13:15–15:00, room L1.5, C-16) and ends on **12 January 2027**, leaving the remaining classes in January as a reserve. The weeks of 17 November and the Christmas break (22 December – 4 January) have no classes; the Christmas break is treated as a time buffer. BK – Bartosz Ksel, KP – Krzysztof Pogoda, MS – Maciej Smolarz, PU – Polona Umek. Effort is the estimated total work of the team in hours.

Task	Description	Period	Resp.	Effort [h]
T1	System analysis, components and events	12.10–19.10	BK	8
T2	Project proposal	12.10–20.10	all	16
T3	Petri net model, EF, next_state	20.10–10.11	BK, KP	30
T4	Safety and recoverability analysis	03.11–01.12	KP	25
T5	Simulator core (plant + controller)	20.10–01.12	MS	40
T6	Fault and service scenarios, GUI	17.11–15.12	PU	30
T7	Prioritization policies, experiments	24.11–15.12	BK, MS	25
T8	Final report and presentation	08.12–12.01	all	35
–	Meetings and coordination	whole project	all	30
			Total	**239**

Milestones: **M1** (20.10.2026) – proposal submitted; **M2** (01.12.2026) – model verified, simulator working; **M3** (12.01.2027) – final report and demonstration.

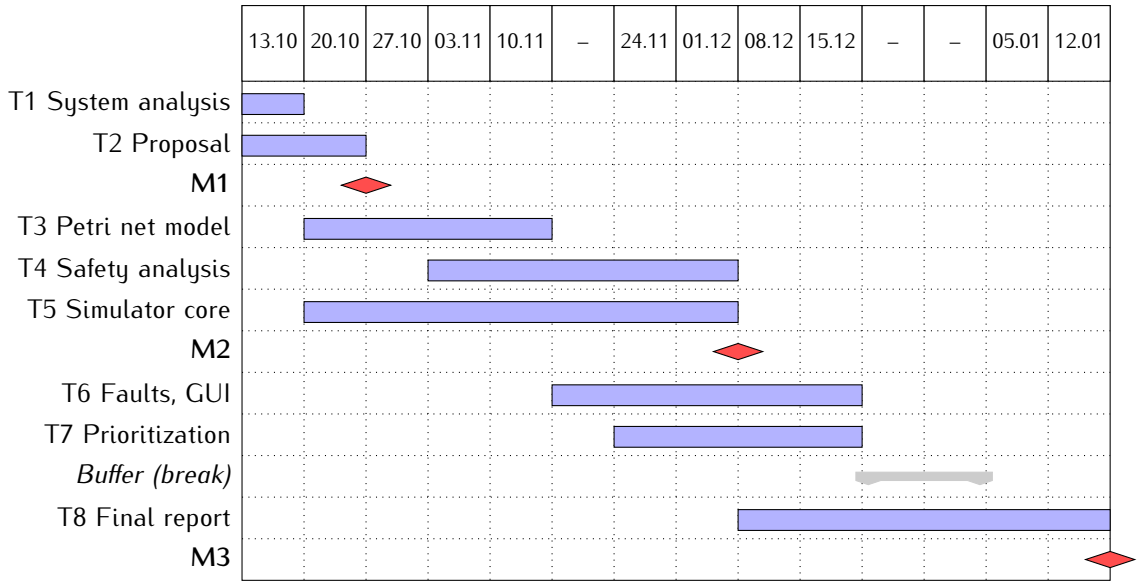


Figure 1: Gantt chart. Columns are weeks, labelled with the date of the class; "–" marks weeks without classes.

4 Delivery

- **M1, 20.10.2026 – proposal report:** problem statement, model and control concept, work plan. Delivered as PDF and hard copy.
- **M2, 01.12.2026 – progress report:** the complete Petri net model, the rules EF(s), next_state(s, e) and safe(s, e), results of the reachability analysis. Attached: model files and an archive with the simulator source code.
- **M3, 12.01.2027 – final report:** full model description, proof or verification of safety and recoverability, description of the prioritization policies and comparison of their efficiency, user guide of the simulator. Attached: archive with the source code, model files and simulation results. Delivered as PDF and hard copy, followed by a live demonstration.

Degree of disclosure: the reports and the code are available to the lecturer and the team; the team agrees to their use for teaching purposes.

5 Project Management

The coordinator plans the work, assigns tasks, keeps the schedule and is the contact person for the lecturer. Each task has one owner responsible for its result. The team meets once a week, right after the Tuesday class (30 minutes), to report progress against the Gantt chart; extra online meetings are called before each milestone. Progress is tracked as issues on the repository board.

All code, model files and the LaTeX sources of the reports are kept in a shared Git repository; changes are merged through pull requests reviewed by another team member. Reports are written jointly in LaTeX (Overleaf synchronised with the repository). Day-to-day communication uses a group chat; decisions made there are recorded in the issue tracker.

Decisions are made by consensus, and if it cannot be reached, by majority vote; in a tie the coordinator decides. Conflicts that cannot be resolved inside the team are brought to the lecturer. The results of the work are the joint property of all team members, and each member may use them with a reference to the team.

6 Team

- **Bartosz Ksel** (275472) – project coordinator; system analysis, Petri net model, prioritization policies.
- **Krzysztof Pogoda** (275431) – Petri net model, safety and recoverability analysis.
- **Maciej Smolarz** (275491)– simulator core, experiments.
- **Polona Umek** (300468) – fault and service scenarios, GUI.

All members take part in writing the reports.

7 Summary

We propose to design and verify an event-based supervisory controller for an automatic espresso machine. The model includes not only the normal brewing cycle but also faults and service actions, which makes it possible to state and demonstrate meaningful safety and recoverability properties and to study how the choice of event priorities affects the efficiency of the machine. The result will be a verified Petri net model and a simulator demonstrating the controller.

References

[1] Christos G. Cassandras and Stéphane Lafortune. *Introduction to Discrete Event Systems*. Springer, 3 edition, 2021.

[2] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.